Материал для самостоятельного чтения (20–30 минут)

Python (IDLE)
Тема: выбор типа чисел (int и float), переменные, ввод input(), ветвление if/else.

---

1) Зачем вообще выбирать тип чисел

В программировании важно заранее понимать, *какие данные у нас есть* и *что мы с ними делаем*. Ошибки часто появляются не потому, что формула сложная, а потому что:

- мы храним число не в том виде (например, целое вместо вещественного);
- неправильно понимаем, что возвращает ввод;
- неверно сравниваем данные;
- пишем условие так, что программа “выбирает” не то, что мы ожидали.

Сегодня вы учитесь делать **осознанный выбор**:

- где нужны **целые** числа (int);
- где нужны **вещественные** (float);
- как программа делает выбор между двумя действиями с помощью if/else.

---

2) Как устроена работа в IDLE (чтобы не путаться)

В IDLE обычно есть два основных окна.

1) Окно редактора (Editor)
Здесь пишут программу: несколько строк кода, сохраняют файл и запускают программу целиком.

2) Окно выполнения (Shell)
Здесь видно:

- вывод print(...);
- ошибки (если они есть);
- можно быстро попробовать отдельные команды.

Запомните простое правило:
код удобно писать в редакторе, а **результат смотреть в Shell**.

---

3) Переменные: что это и зачем

Переменная — это имя, к которому “прикреплено” значение.
Это как подпись на коробке: вы кладёте туда данные и потом обращаетесь по имени.

Присваивание делается знаком =:

pythonCopy

```
a = 25

name = "Ivan"

pi = 3.14
```

- a хранит число 25
- name хранит текст
- pi хранит число с дробной частью

Важно: = не означает “равно” в математическом смысле

В программировании = означает: **помести значение справа в переменную слева**.

Пример:

pythonCopy

```
x = 5

x = x + 1
```

Это читается так: “возьми текущее значение x, прибавь 1 и запиши обратно в x”.
После этого x станет 6.

---

4) Типы данных: на что смотрит Python

Python умеет хранить разные типы данных. Сегодня нам важны:

- int — **целые числа**: -2, -1, 0, 1, 2, 3...
- float — **вещественные числа** (с дробной частью): 3.14, 0.5, -2.75
- str — **строка (текст)**: "Привет", "25", "3.14"
- bool — **логическое значение**: True или False

Как понять разницу между числом и строкой

Сравните:

- 25 — число (можно прибавлять, умножать как число)
- "25" — текст (это просто символы “2” и “5”)

Например, в Python:

- 25 + 5 даст 30
- "25" + "5" даст "255" (склеивание текста)

Это ключевой момент, потому что input() возвращает строку.

---

5) int и float: что выбрать и почему

int (целые) выбираем, когда:

- считаем **количество**: учеников, страниц, предметов, шагов;
- результат по смыслу **не может** быть дробным;
- мы ожидаем значения типа “5 штук”, “12 человек”.

Примеры ситуаций:

- “Сколько учеников в классе?” → int
- “Сколько раз ученик повторил упражнение?” → int

float (вещественные) выбираем, когда:

- есть измерения и возможна дробь: цена, масса, расстояние, время, температура;
- результат вычислений может получиться нецелым (например, среднее значение).

Примеры ситуаций:

- “Цена 19.99” → float
- “Расстояние 2.5 км” → float
- “Средний балл” → float

Частая ошибка

Иногда ученики думают: “Если число выглядит целым, значит это int”.
Но важно не то, как оно *выглядит*, а то, **может ли в задаче появиться дробная часть**.

Например:

- “Температура сегодня 2 градуса” — выглядит целым, но завтра может быть 2.5 → лучше float для температуры.
- “Количество учеников сегодня 25” — и завтра тоже 25, дроби не бывает → int.

---

6) Ввод данных: что делает input()

Команда input() получает данные от пользователя:

pythonCopy

```
text = input("Введите слово: ")

print(text)
```

Главное правило: input() всегда возвращает **строку** (str).
Даже если вы ввели число.

Пример:

pythonCopy

```
age = input("Введите возраст: ")

print(age)
```

Если пользователь введёт 15, переменная age будет хранить "15" (строку).

Зачем это важно

Потому что сравнение и вычисления зависят от типа данных.
Если вы хотите считать, а у вас строка — программа поведёт себя неожиданно.

---

7) Ветвление: как программа “принимает решение”

В реальной жизни мы часто действуем по условию:

- если идёт дождь → беру зонт
- иначе → не беру

В программировании это записывается через if/else.

Шаблон:

pythonCopy

```
if условие:
    действие_1
else:
    действие_2
```

- if проверяет условие.
- Если условие **истинно** (True), выполняется блок if.
- Если условие **ложно** (False), выполняется блок else.

Пример со строкой (как в учебных примерах)

pythonCopy

```
bird = input("Какая птица розовая? ")


if bird == "Розовый фламинго":
    print("Верно!")
else:
    print("Неверно! Это розовый фламинго.")
```

---

8) Сравнение: == и что оно означает

Чтобы проверить “равно ли”, используют ==.

- = — присваивание
- == — сравнение

Пример:

pythonCopy

```
x = 10
if x == 10:
    print("x равно 10")
```

Внутри if должно быть **условие**, то есть выражение, которое даёт True или False.

---

9) Отступы — это часть синтаксиса Python

Python определяет блоки кода по **отступам**.

Правильно:

pythonCopy

```
if True:
    print("Да")
else:
    print("Нет")
```

Неправильно (смешанные или отсутствующие отступы) приводит к ошибкам или к неверной логике.

Обычно используют **4 пробела** на каждый уровень вложенности.

---

10) Как читать и объяснять программу (навык, который вам нужен)

Когда вы видите код, пробуйте проговорить его смысл:

Пример:

pythonCopy

```
bird = input("Какая птица розовая? ")
if bird == "Розовый фламинго":
    print("Верно!")
else:
    print("Неверно! Это розовый фламинго.")
```

Как объяснить словами:

1. программа спрашивает у пользователя вопрос и сохраняет ответ в переменную bird;
2. проверяет: равен ли ответ точной строке "Розовый фламинго";
3. если равен — печатает “Верно!”;
4. иначе — печатает подсказку, что правильно.

---

11) Типичные ошибки и как их избежать

1. **Путают = и ==**

- x = 5 — записали 5 в переменную
- x == 5 — проверили, равно ли

2. **Забывают кавычки у строк**

- правильно: "Розовый фламинго"
- неправильно: Розовый фламинго

3. **Пробелы и отступы**
   Отступы обязательны. Если отступы “скачут”, программа может не запуститься.

4. **Ожидают число, а получают строку**
   input() даёт строку. Поэтому нужно помнить о типах при сравнении и вычислениях.

---

12) Итог (что вы должны уметь после чтения)

Вы должны уверенно отвечать на вопросы:

- Чем int отличается от float?
- В каких задачах нужен float, даже если число иногда выглядит целым?
- Что возвращает input()?
- Как работает if/else и что такое условие?
- Почему в Python важны отступы?